# Development of Flutter Apps for Fourier Transform

## With Symbolic Derivation and Numerical Visualisation: Supported by a Rule-First Python Backend

Flutter application as the main project focus

Supporting backend used for symbolic Fourier derivation

# Table of Contents

# Abstract

This dissertation presents the development of a Flutter application for Fourier transform learning on smart devices. The original project requirement focuses on Flutter programming, simulator or device testing, step-by-step workings, and graphical illustrations of relevant Fourier transform equations. The project therefore keeps the Flutter application as the main user-facing system.

During development, a Flutter-only symbolic processing direction was considered. Flutter and Dart were suitable for user interaction, local numerical experiments, and visual presentation, but they were not sufficient for reliable symbolic Fourier transform derivation involving distribution-theory objects such as Dirac delta, Heaviside step functions, principal value terms, and engineering-style LaTeX steps. For this reason, the final design introduces a Python FastAPI backend as a supporting symbolic computation service.

The completed system combines a Flutter learning interface, structured examples, LaTeX display, numerical visualisation, and a rule-first backend for symbolic derivation. Current validation includes 68 passing backend tests and 6 passing Flutter tests, with 1 optional frontend-backend e2e test skipped unless explicitly enabled.

# Introduction

The project was originally defined as the development of Flutter apps running on smart devices for Fourier transform. The expected application should be tested on a simulator or physical device, show step-by-step workings, and provide graph plots or illustrative graphics where useful.

This requirement makes the Flutter app the centre of the project. The backend introduced later should therefore be understood as a supporting service for symbolic derivation rather than as a replacement for the Flutter application objective.

The educational motivation is that students learning Fourier transform need to see both the final frequency-domain result and the reasoning that leads to it. A useful learning app should therefore support expression input, readable formulas, guided examples, derivation steps, and visual intuition.

# Background and Design Rationale

## Fourier Transform Background

The project uses the engineering Fourier transform convention

```
X(omega) = integral x(t) exp(-j omega t) dt
```

The display layer uses the engineering imaginary unit j and treats omega as real angular frequency. Distribution-theory objects such as delta, unit step, Cauchy principal value, and sign functions are used where ordinary integrals are not sufficient.

### Motivation

General-purpose computer algebra systems can compute many symbolic transforms, but their output is often not aligned with engineering teaching notation. Results may contain Piecewise branches, RootSum expressions, complex assumptions, or other symbolic structures that are mathematically valid but difficult for students to interpret.

```
CAS-style issue:
Piecewise(... complex conditions ...)

Engineering textbook target:
F{1/(t^2+1)} = pi * exp(-abs(omega))
```

### Project Objectives

- Develop a Flutter frontend for expression input, examples, LaTeX display, and graph-based learning.
- Provide step-by-step workings for supported Fourier transform questions.
- Explain why a Flutter-only symbolic processing approach was not sufficient for the final derivation target.
- Use a Python backend to support distribution-aware symbolic derivation.
- Validate backend mathematical output and frontend layout behaviour with automated tests.

### Scope and Limitations

The app is not designed as an unrestricted computer algebra system. It supports a defined engineering range, including constants, impulses, steps, trigonometric functions, exponentials, PV reciprocal forms, polynomial distributions, polynomial-step forms, selected rational forms, finite windows, and linear combinations of supported terms.

## Overall System Architecture

### System Overview

The system is best explained through responsibilities rather than a complex flow diagram. Flutter handles user-facing learning tasks, while the Python backend handles symbolic derivation. The two layers communicate through a JSON API.

| Layer | Main responsibility |
|---|---|
| Flutter application | Smart-device interface, keypad input, structured examples, LaTeX display, local numerical visualisation, and small-screen layout handling. |
| FastAPI symbolic backend | Expression parsing, rule-first symbolic derivation, distribution theory rules, result_latex formatting, and steps_latex generation. |

| | |
|---|---|
| Communication boundary | The Flutter app sends user expressions to /fourier and receives input_latex, result_latex, and steps_latex. |

### Flutter Frontend

The frontend is responsible for input, examples, formula preview, result display, numerical visualisation, and responsive layout. It remains the main application layer because the project requirement is specifically focused on Flutter apps for Fourier transform.

### FastAPI Symbolic Backend

The backend is a supporting symbolic service. It was introduced because a reliable symbolic Fourier transform engine requires expression parsing, algebraic rewriting, distribution-theory rules, and controlled LaTeX output.

### Frontend-Backend Communication

The frontend sends the actual user expression to the backend. This shows that the app is not limited to fixed examples; it can process general user input within the supported rule range.

```
final res = await http.post(
  uri,
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({'expression': expression}),
);
```

## Flutter Application Design

### User Interface

The home page provides expression input, formula preview, example navigation, FFT controls, and access to symbolic and numerical result views.

### Formula Input

The keypad and expression handling support common signal expressions such as u(t), delta(t), sin, cos, exp, powers, and fractions. The app also includes local numerical handling for visualisation.

### Example Navigation

The example list was expanded from simple transform pairs into a structured set of teaching examples. Each example stores the expression, title, category, input LaTeX, transform object, and description.

```
class FourierExample {
  final String expression;
  final String title;
  final String category;
```

```
  final String inputLatex;
  final String transformLatex;
  final String description;
}
```

### LaTeX Rendering

The frontend uses LaTeX rendering for formula preview, symbolic results, derivation steps, and formula references. Long formulas and notes are handled through local scroll containers so that narrow screens do not cause overflow.

```
class ScrollableMathLine extends StatefulWidget {
  final String latex;
  final TextStyle? textStyle;
  final String? semanticsLabel;
}
```

## Symbolic Fourier Transform Engine

### Definition-Based Framework

Every teaching derivation is grounded in the Fourier transform definition. The backend may use known pairs and properties, but the displayed derivation is written as a mathematical explanation rather than a trace of internal matcher logic.

### Rule Matching Strategy

The backend applies rules before falling back to less controlled symbolic forms. This keeps the output predictable and closer to engineering textbook notation.

```
rules = [
    exact_matches,
    known_transform_pairs,
    distribution_rules,
    property_based_rules,
    decomposition_rules,
    controlled_fallback,
]
```

### Distribution-Based Transforms

Distribution-based transforms are a key reason for using a Python backend. For example, polynomial-step signals require delta derivatives and PV terms.

```
def _rule_poly_times_step_distribution(f):
    if not (isinstance(f, Mul) and f.has(Heaviside)):
        return None

    def _basis(n):
        if n == 0:
            return pi*DiracDelta(omega) - I*(1/omega)
        return (
```

```
            pi*(I**n)*Derivative(DiracDelta(omega), (omega, n))
            + ((-1)**n)*(I**(n-1))*factorial(n)*PV(1/(omega**(n+1)))
        )
```

### General Transform Families

The engine supports parameterized transform families rather than only fixed examples. These include shifted impulses, shifted steps, sinusoids with frequency and phase, complex exponentials, one-sided exponentials, PV reciprocal forms, polynomial distributions, polynomial-step forms, selected rational forms, finite windows, and linear combinations.

### Error Handling

If an expression falls outside the supported range, the backend should return a controlled fallback or error instead of pretending to have a reliable closed-form result.

## Implementation Details

### Backend Commands and Functions

The backend symbolic API is implemented through the /fourier endpoint. The response contract explicitly separates the original input, final transform result, and derivation steps.

```
class FourierResponse(BaseModel):
    ok: bool
    input_latex: str
    result_latex: str
    steps_latex: list[str]
    error: str | None = None
```

### Key Code Snippets

The following function prepares derivation steps for teaching display by filtering internal implementation details and unwanted symbolic artifacts.

```
def _teaching_steps(f, X, steps):
    cleaned = []
    for raw in steps or []:
        s = _format_step_display_latex(str(raw).strip())
        if 'Method:' in s or '_rule_' in s or 'matcher' in s:
            continue
        if 'Piecewise' in s or 'RootSum' in s or 'meijerg' in s:
            continue
        cleaned.append(s)
    return cleaned
```

### Matcher Design

The matcher design is rule-first. This means exact and known transform families are attempted before fallback strategies. The design makes the supported range explicit and testable.

### API Design

The API is intentionally simple: the frontend sends an expression string and receives LaTeX-ready fields. This design directly supports the Flutter UI and avoids exposing backend-specific symbolic objects to the frontend.

### Data Flow

The data flow is: user expression in Flutter, POST request to /fourier, backend parsing and rule matching, LaTeX result generation, and Flutter rendering. This sequence is described in prose rather than as a diagram to keep the dissertation logic clear.

## Testing and Evaluation

### Backend Regression Tests

```
cd backend
.\.venv\Scripts\python.exe -m pytest tests -q

Result: 68 passed
```

Backend tests cover strict result_latex checks, teaching-style steps_latex checks, representative step snapshots, and output guards against Piecewise, RootSum, arg, polar_lift, meijerg, matcher, debug, and srepr.

### Flutter Tests

```
cd flutter_app
flutter test

Result: 6 passed, 1 skipped
```

Flutter tests cover responsive breakpoints, home page rendering, example navigation, multiple screen sizes without layout exceptions, local scroll containers for long formulas and notes, and FT Steps long-formula display on narrow screens.

### Representative Test Cases

Representative backend step cases include 1, delta(t-3), u(t), t*u(t), sin(t)*u(t), frac(1,3*t-2), frac(1,t^2+1), and u(t-2)-u(t-5). These cases check both mathematical result quality and teaching-step clarity.

### Performance Observations

Local numerical visualisation remains in Flutter for responsiveness, while symbolic derivation is handled by the backend. This separation avoids network latency for interactive chart updates and keeps exact symbolic work in the more suitable Python environment.

# Discussion

## Current Capabilities

The app supports general user input within a defined engineering range. It is not limited to fixed examples, although the example list helps guide users toward supported expression families.

## Supported Function Classes

| Function class | Examples |
|---|---|
| Impulses and steps | delta(t), delta(t-a), u(t), u(t-a) |
| Trigonometric and exponential | sin(a*t+b), cos(a*t+b), exp(j*w0*t), exp(-a*t)u(t) |
| Distribution and polynomial | PV(1/t), 1/(a*t+b), t^n, t^n u(t) |
| Rational and window forms | 1/(t^2+a^2), t/(t^2+a^2), u(t-a)-u(t-b) |

## Limitations

The system is not an unrestricted CAS. Unsupported expressions may return an integral form or error. This limitation is acceptable because the project prioritises explainable and testable engineering output.

## Comparison with General CAS Systems

General CAS systems prioritise symbolic completeness. This project prioritises educational readability, engineering notation, and step-by-step reasoning. The rule-first method provides cleaner output for the supported range but requires deliberate rule expansion.

# Conclusion and Future Work

This project developed a Flutter-based Fourier transform learning app with symbolic derivation and numerical visualisation. The project remains aligned with the original Flutter smart-device app requirement, while the Python backend provides supporting symbolic computation where Flutter-only processing is not sufficient.

Future work includes expanding rule coverage, modularising the backend, improving unsupported-case explanations, adding screenshot-based visual regression tests, and supporting exportable derivation reports for classroom use.

## References


- Course materials and standard Signals and Systems Fourier transform tables.
- Flutter documentation for application development and widget testing.
- FastAPI documentation for backend API implementation.
- SymPy documentation for symbolic parsing and algebraic manipulation.


## Appendix A. Question & Answer Summary

A detailed Q&A file is maintained in docs/QA.md, and a shorter email version is maintained in docs/QA_email_brief.md. These files answer the supervisor's concerns about general user input capability, generic transform support within scope, PV explanation, examples, LaTeX display, derivation steps, testing, and dissertation formatting.

## Appendix B. Additional Code Snippets

Longer implementation details, such as full rule functions, local numerical FFT code, parser details, and deployment configuration, should remain in the source repository or appendix rather than the main dissertation body.

## Appendix C. Example Inputs and Outputs

The test data reference is maintained in test.md. It includes implemented transform cases and future/pending transform pairs for later rule expansion.